

ΠΡΟΧΩΡΗΜΕΝΑ ΘΕΜΑΤΑ
ΤΕΧΝΟΛΟΓΙΑΣ ΚΑΙ ΕΦΑΡΜΟΓΩΝ
ΒΑΣΕΩΝ ΔΕΔΟΜΕΝΩΝ

ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΕΡΓΑΣΙΑ ΓΙΑ
ΤΟ ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2025-2026

ΟΜΑΔΑ

ΠΑΝΑΓΙΩΤΗΣ ΣΤΑΥΡΟΠΟΥΛΟΣ
4990

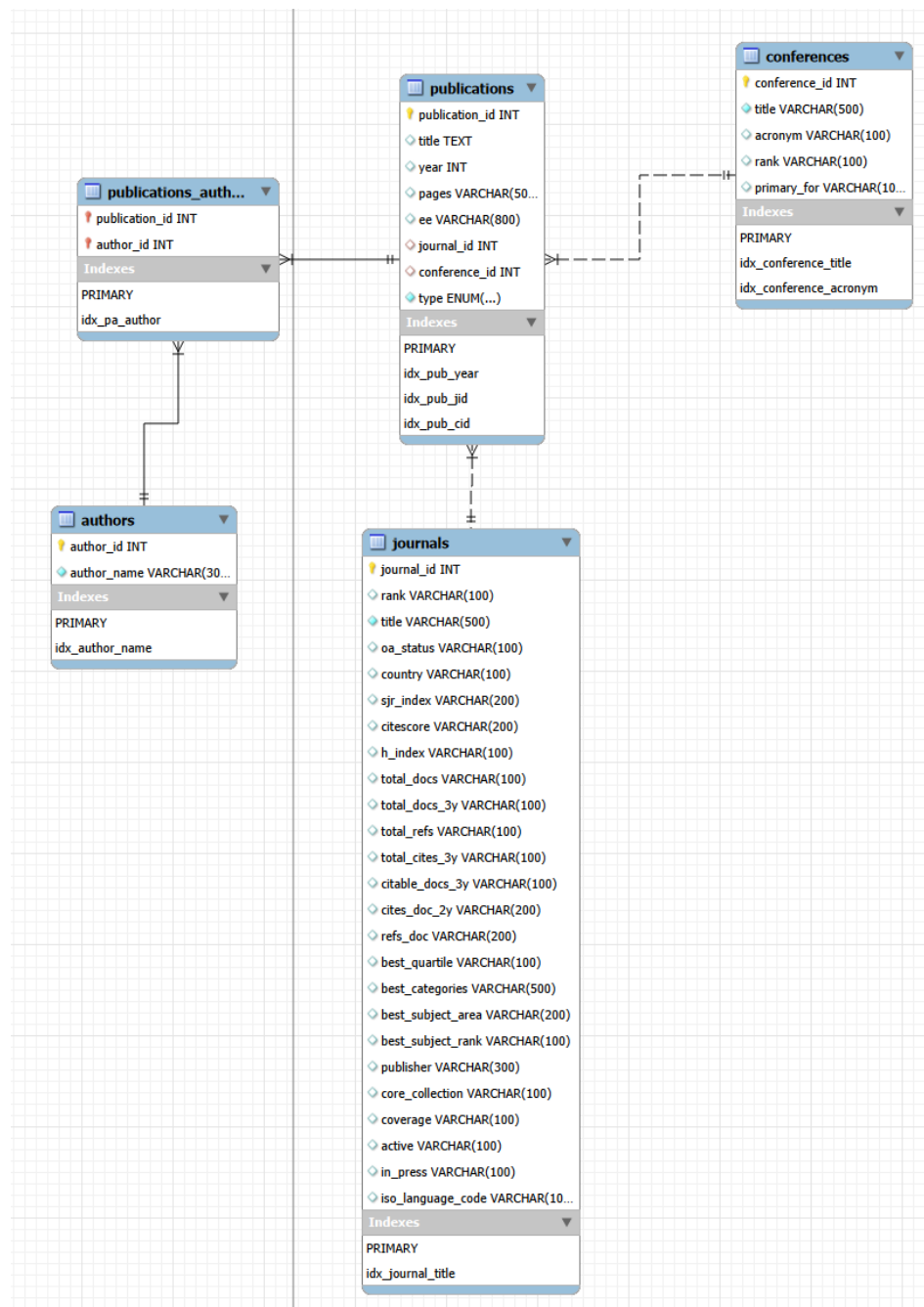
ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

ΜΑΪΟΣ 2026

1 ΒΑΣΗ ΔΕΔΟΜΕΝΩΝ

Η βάση δεδομένων του συστήματος ονομάζεται `data_visualizer` και περιέχει δεδομένα σχετικά με άρθρα περιοδικών/συνεδρίων (από το DBLP dataset) και κάποιες μετρικές αξιολόγησης από άλλες πηγές (πχ iCore26).

1.1 ΣΧΕΣΙΑΚΟ ΣΧΗΜΑ ΣΕ ΛΟΓΙΚΟ ΕΠΙΠΕΔΟ



Σχήμα 1.1 Σχεσιακό σχήμα της βάσης δεδομένων του συστήματος

Section-break (continuous)

```

SET
@OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_C
HECKS, FOREIGN_KEY_CHECKS=0;

-----
-- Schema: Data Visualizer
-----

CREATE DATABASE IF NOT EXISTS
`data_visualizer`;
USE `data_visualizer`;

-----
-- Table `authors`
-----

CREATE TABLE `authors` (
  `author_id` INT NOT NULL
  AUTO_INCREMENT,
  `author_name` VARCHAR(300) NOT
  NULL,
  PRIMARY KEY (`author_id`),
  INDEX `idx_author_name`
  (`author_name`)
) ENGINE = InnoDB;

-----
-- Table `journals`
-----

CREATE TABLE `journals` (
  `journal_id` INT NOT NULL
  AUTO_INCREMENT,
  `rank` VARCHAR(100) NULL,
  `title` VARCHAR(500) NOT NULL,
  `oa_status` VARCHAR(100) NULL,
  `country` VARCHAR(100) NULL,
  `sjr_index` VARCHAR(200) NULL,
  `citescore` VARCHAR(200) NULL,
  `h_index` VARCHAR(100) NULL,
  `total_docs` VARCHAR(100) NULL,
  `total_docs_3y` VARCHAR(100) NULL,
  `total_refs` VARCHAR(100) NULL,
  `total_cites_3y` VARCHAR(100) NULL,
  `citable_docs_3y` VARCHAR(100)
  NULL,
  `cites_doc_2y` VARCHAR(200) NULL,
  `refs_doc` VARCHAR(200) NULL,
  `best_quartile` VARCHAR(100) NULL,
  `best_categories` VARCHAR(500)
  NULL,
  `best_subject_area` VARCHAR(200)
  NULL,
  `best_subject_rank` VARCHAR(100)
  NULL,
  `publisher` VARCHAR(300) NULL,
  `core_collection` VARCHAR(100)
  NULL,
  `coverage` VARCHAR(100) NULL,
  `active` VARCHAR(100) NULL,
  `in_press` VARCHAR(100) NULL,
  `iso_language_code` VARCHAR(100)
  NULL,
  PRIMARY KEY (`journal_id`),
  INDEX `idx_journal_title`
  (`title`(500))
) ENGINE = InnoDB;

```

```

-----
-- Table `conferences`
-----

CREATE TABLE `conferences` (
  `conference_id` INT NOT NULL
  AUTO_INCREMENT,
  `title` VARCHAR(500) NOT NULL,
  `acronym` VARCHAR(100) NULL,
  `rank` VARCHAR(100) NULL,
  `primary_for` VARCHAR(100) NULL,
  PRIMARY KEY (`conference_id`),
  INDEX `idx_conference_title`
  (`title`(500)),
  INDEX `idx_conference_acronym`
  (`acronym`)
) ENGINE = InnoDB;

-----
-- Table `publications`
-----

CREATE TABLE `publications` (
  `publication_id` INT NOT NULL,
  `title` TEXT NULL,
  `year` INT NULL,
  `pages` VARCHAR(500) NULL,
  `ee` VARCHAR(800) NULL,
  `journal_id` INT NULL,
  `conference_id` INT NULL,
  `type` ENUM('journal', 'conference')
  NOT NULL,
  PRIMARY KEY (`publication_id`),
  CONSTRAINT `journal_fk` FOREIGN KEY
  (`journal_id`) REFERENCES
  `journals`(`journal_id`),
  CONSTRAINT `conference_fk` FOREIGN
  KEY (`conference_id`) REFERENCES
  `conferences`(`conference_id`)
) ENGINE = InnoDB;

-----
-- Table `publications_authors`
-----

CREATE TABLE `publications_authors` (
  `publication_id` INT NOT NULL,
  `author_id` INT NOT NULL,
  PRIMARY KEY (`publication_id`,
  `author_id`),
  CONSTRAINT `publication_fk` FOREIGN
  KEY (`publication_id`) REFERENCES
  `publications`(`publication_id`),
  CONSTRAINT `author_fk` FOREIGN KEY
  (`author_id`) REFERENCES
  `authors`(`author_id`)
) ENGINE = InnoDB;

-----
-- Staging Table:
`staging_publications`
-----

CREATE TABLE IF NOT EXISTS
`staging_publications` (
  `id` INT,
  `title` TEXT,
  `year` INT NULL,

```

```

`pages` TEXT,
`ee` TEXT,
`journal` VARCHAR(500),
`booktitle` VARCHAR(500),
`type` TEXT
) ENGINE = InnoDB;

-----
-- Staging Table:
`staging_publications_authors`
-----

CREATE TABLE IF NOT EXISTS
`staging_publications_authors` (
  `publication_id` INT,
  `author_name` VARCHAR(300)
) ENGINE = InnoDB;

SET
FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHE
CKS;

Section-break (continuous)

```

1.2 ΣΧΕΣΙΑΚΟ ΣΧΗΜΑ ΣΕ ΦΥΣΙΚΟ ΕΠΙΠΕΔΟ

1.2.1 ΡΥΘΜΙΣΗ ΤΩΝ ΠΑΡΑΜΕΤΡΩΝ ΤΟΥ DBMS

- Όλοι οι πίνακες χρησιμοποιούν InnoDB ως storage engine.
- Για την φόρτωση των δεδομένων αυξήσαμε προσωρινά το innodb_buffer_pool_size σε 8GB με την εντολή:

“SET GLOBAL innodb_buffer_pool_size = 8589934592;”.

Χωρίς αυτή την ρύθμιση, η MySQL θα ξεφόρτωνε συνεχώς σελίδες στον δίσκο κατά τα bulk insertions, γεγονός το οποίο θα καθιστούσε την φόρτωση 7M εγγραφών στον πίνακα publications_authors πολύ πιο αργή. Η αλλαγή είναι προσωρινή και επαναφέρεται αυτόματα.

1.2.2 ΡΥΘΜΙΣΗ ΤΟΥ ΦΥΣΙΚΟΥ ΣΧΗΜΑΤΟΣ ΤΗΣ ΒΑΣΗΣ ΔΕΔΟΜΕΝΩΝ

Indexes:

Έχουμε ορίσει indexes σε 2 φάσεις. Η 1^η φάση είναι κατά την φόρτωση, για να επιταχύνουμε τα όποια joins. Τα indexes που αντιστοιχούν σε αυτήν την φάση είναι τα:

Index	Table	Column
idx_author_name	authors	author_name
idx_journal_title	journals	title
idx_conference_title	conferences	title
idx_conference_acronym	conferences	acronym

Η 2^η φάση είναι μετά την φόρτωση, όπου έχουμε ορίσει indexes, για να επιταχύνουν τα queries της εφαρμογής. Τα αντίστοιχα indexes είναι τα:

Index	Table	Column
idx_pub_year	publications	year
idx_pub_jid	publications	journal_id
idx_pub_cid	publications	conference_id
idx_pa_author	publications_authors	author_id

Ο λόγος που φτιάξαμε αυτά τα indexes μετά την φόρτωση και όχι πριν, είναι γιατί αν τα είχαμε φτιάξει απο πριν, μετά απο κάθε bulk insert, θα έπρεπε να γίνει ενημέρωση του B+ δέντρου, γεγονός που θα καθυστερούσε αρκετά για εκατομύρια εγγραφές.

Views:

Έχουμε ορίσει 9 views με στόχο την απλοποίηση των stored procedures.

Αναλυτικότερα:

1. `author_publications_view`: Συνδέει τους πίνακες `publications`, `publications_authors`, `authors`, `journals`, `conferences`. Επιστρέφει για κάθε συγγραφέα όλες τις δημοσιεύσεις του με το όνομα του `venue` επιλυμένο (ή Unknown). Χρησιμοποιείται από όλες τις procedures του Author tab
2. `venue_publications_view`: Ενοποιεί περιοδικά και συνέδρια σε ένα κοινό σχήμα με UNION ALL, ώστε η αναζήτηση `venues` να γίνεται με ένα ερώτημα ανεξάρτητα από τον τύπο. Χρησιμοποιείται για αναζήτηση και στατιστικά του Venue tab
3. `venue_author_publications_view`: Παρόμοια με την προηγούμενη αλλά περιλαμβάνει και τα `author_id`, ώστε να μπορούν να υπολογιστούν στατιστικά σχετικά με συγγραφείς ανά `venue` (π.χ. μέσος αριθμός συγγραφέων ανά άρθρο)
4. `valid_publications_view`: Απλό φίλτρο πάνω στον πίνακα `publications` που κρατά μόνο εγγραφές με όχι null έτος. Χρησιμοποιείται από τις procedures του Year tab για να αποφευχθούν εγγραφές χωρίς έτος
5. `publisher_stats_view`: Ομαδοποιεί τα περιοδικά ανά εκδότη και υπολογίζει τον αριθμό τους με κατανομή Q1, Q2, Q3, Q4. Τροφοδοτεί το StackedBarChart του Charts tab
6. `journal_metrics_view`: Επιλέγει πεδία από τον πίνακα `journals` (SJIR, CiteScore, h-index κ.λπ.) φιλτράροντας εγγραφές με κενές τιμές. Τροφοδοτεί το ScatterChart του Charts tab
7. `year_publication_details_view`: Συνδέει `publications`, `journals`, `conferences`, `publications_authors` και `authors`. Επιστρέφει δημοσιεύσεις ανά έτος συμπεριλαμβανομένων των ονομάτων συγγραφέων, ώστε να υποστηρίζεται το φιλτράρισμα ανά συγγραφέα στο Year tab
8. `journal_category_year_view`: Συνδέει `publications` με `journals` και εξάγει για κάθε δημοσίευση το έτος, το `venue id` και το `subject area`. Χρησιμοποιείται για το LineChart κατηγοριών περιοδικών στο Charts tab
9. `conference_category_year_view`: Αντίστοιχο της προηγούμενης για συνέδρια, χρησιμοποιώντας το πεδίο `primary_for` ως κατηγορία

1.2.3 ΡΥΘΜΙΣΗ ΑΣΦΑΛΕΙΑΣ

Για την εφαρμογή δημιουργήσαμε έναν νέο user, αναλυτικότερα:

```
CREATE USER 'DataVisualizerUser'@'localhost' IDENTIFIED BY  
'DataVisualizer';
```

```
GRANT ALL PRIVILEGES ON `data_visualizer`.* TO  
'DataVisualizerUser'@'localhost';
```

```
GRANT SYSTEM_VARIABLES_ADMIN ON *.* TO  
'DataVisualizerUser'@'localhost';
```

```
GRANT FILE ON *.* TO 'DataVisualizerUser'@'localhost'
```

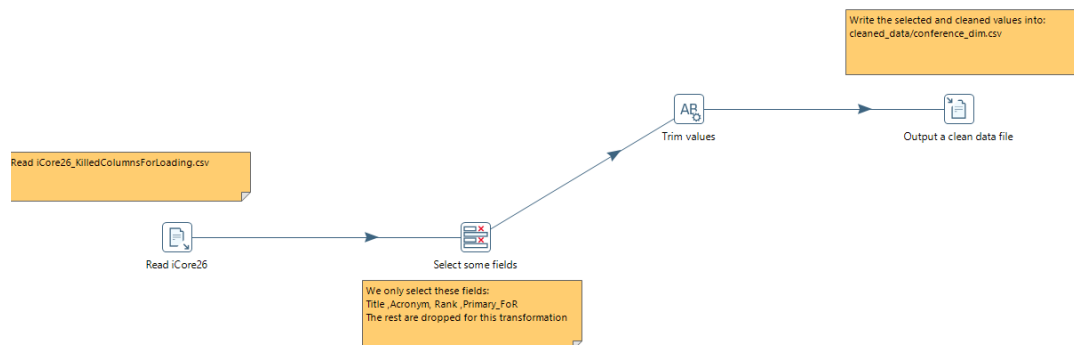
Ο χρήστης αυτός, έχει πλήρη δικαιώματα μόνο στην βάση data_visualizer και όχι στο υπόλοιπο σύστημα. Τα δικαιώματα SYSTEM_VARIABLES_ADMIN και FILE είναι απαραίτητα για την φόρτωση των δεδομένων (για την αλλαγή του innodb_buffer_pool_size, αλλά και για την εντολή LOAD DATA INFILE).

2 ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΛΟΓΙΣΜΙΚΟΥ

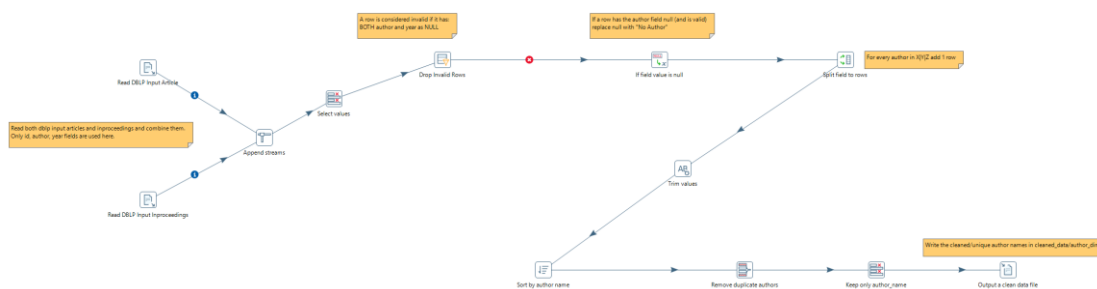
2.1 ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΚΑΙ ΔΟΜΗ ETL

Για την προεπεξεργασία και φόρτωση των δεδομένων χρησιμοποιήσαμε το Pentaho Data Integration (Spoon). Φτιάξαμε 5 transformations, ένα για κάθε πίνακα και ένα κεντρικό job (prepare_and_clean_data.kjb) που τα εκτελεί όλα διαδοχικά. Κάθε transformation, διαβάζει τα raw αρχεία, κρατάει τις στήλες που χρειαζόμαστε, φιλτράρει μη έγκυρες εγγραφές και γράφει το αποτέλεσμα στα clean CSVs. Τα 5 cleaned CSVs που παράγονται φορτώνονται στην συνέχεια στην βάση με LOAD DATA INFILE.

- **prepare data for conference dim:** Το transformation αυτό, διαβάζει το αρχείο iCore26_KilledColumnsForLoading.csv, το οποίο περιέχει την κατάταξη συνεδρίων από το iCore26. Από τις διαθέσιμες στήλες κρατάμε μόνο τις Title, Acronym, Rank και PrimaryFoR, κάνουμε trim τις τιμές για να αφαιρεθούν κενά, και γράφουμε το αποτέλεσμα στο conference_dim.csv



- **prepare data for author dim:** Διαβάζει και τα 2 DBLP αρχεία, τα ενώνει και κρατάει μόνο τα πεδία id, author και year. Φιλτράρονται εγγραφές που έχουν ταυτόχρονα null author και null year. Αν μόνο το author είναι null, αντικαθίσταται με "No Author". Στην συνέχεια το πεδίο author, που μπορεί να περιέχει πολλά όνομα χωρισμένα με | σπάει σε ξεχωριστές γραμμές και αφαιρούνται τα διπλότυπα. Το αποτέλεσμα γράφεται στο author_dim.csv



- **prepare data for journal dim:** Διαβάζει το `journal_ranking_data_raw.csv` το οποίο περιέχει μετρικές για περιοδικά. Φιλτράρονται εγγραφές χωρίς τίτλο και κρατάμε μόνο τα πεδία που χρειαζόμαστε. Το αποτέλεσμα γράφεται στο `journal_dim.csv`

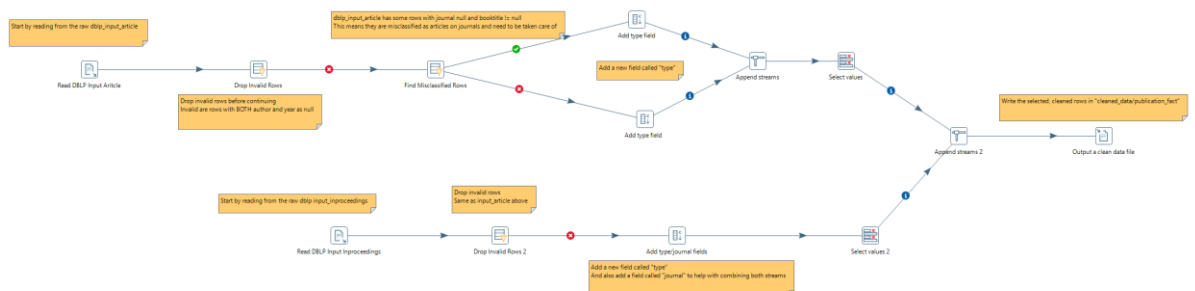


- **prepare data for publication fact:** Διαβάζει και τα δύο DBLP αρχεία ξεχωριστά, καθώς αντιπροσωπεύουν διαφορετικούς τύπους δημοσιεύσεων, το `input_article.csv` περιέχει άρθρα σε journals και το `input_inproceedings.csv` περιέχει δημοσιεύσεις σε συνέδρια. Για κάθε αρχείο φιλτράρονται πρώτα οι άκυρες εγγραφές, δηλαδή γραμμές που έχουν ταυτόχρονα `null id` και `null title`.

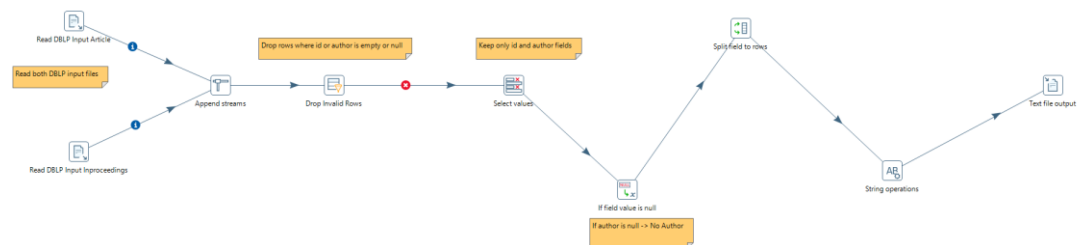
Στο `input_article.csv` υπάρχει ένας επιπλέον έλεγχος: κάποιες εγγραφές έχουν `journal=null` και `booktitle≠null`, που σημαίνει ότι είναι λανθασμένα καταχωρημένες ως journal articles ενώ στην πραγματικότητα είναι conference papers. Αυτές εντοπίζονται από το βήμα "Find Misclassified Rows" και χωρίζονται σε δύο διαφορετικά paths, τα έγκυρα άρθρα παίρνουν `type="journal"` και τα misclassified παίρνουν `type="conference"`.

Για το `input_inproceedings.csv` η διαδικασία είναι πιο απλή, προστίθεται απευθείας `type="conference"` και ένα κενό journal field για να ταιριάζει η δομή των δύο streams.

Στο τέλος όλα τα streams ενώνονται, κρατάμε μόνο τα πεδία `id`, `title`, `year`, `type` και `journal`, και το αποτέλεσμα γράφεται στο `publication_fact.csv`.



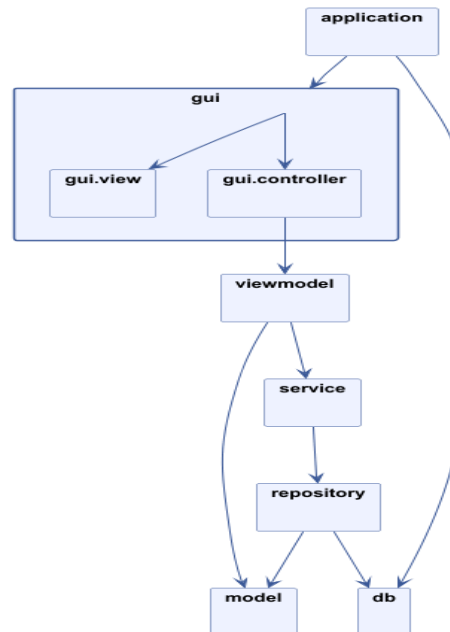
- **prepare data for publication author:** Διαβάζει και τα δύο DBLP αρχεία και τα ενώνει. Φιλτράρονται εγγραφές χωρίς id ή author και κρατάμε μόνο αυτά τα δύο πεδία. Το πεδίο author μπορεί να περιέχει πολλαπλά ονόματα διαχωρισμένα με | , οπότε σπάει σε ξεχωριστές γραμμές, μία γραμμή ανά συγγραφέα ανά δημοσίευση. Το αποτέλεσμα γράφεται στο publication_author.csv, που αποτελεί τον πίνακα γέφυρα μεταξύ publications και authors.



Τέλος, το job που καλεί όλα τα transformation διαδοχικά, είναι της μορφής:



2.2 ΔΙΑΓΡΑΜΜΑΤΑ ΠΑΚΕΤΩΝ / ΥΠΟΣΥΣΤΗΜΑΤΩΝ ΚΕΝΤΡΙΚΗΣ ΕΦΑΡΜΟΓΗΣ



Η εφαρμογή ακολουθεί το MVVM + Service + Repository pattern και έχουμε 6 πακέτα:

1. **gui**: Περιέχει τα FXML αρχεία (Views) και τα αντίστοιχα Controllers. Τα Controllers χειρίζονται όλα τα πεδία FXML στα Views και τα «αναθέτουν» στα αντίστοιχα ViewModels μετά από κάποια ενέργεια του χρήστη
2. **viewmodel**: Υπεύθυνα για την κατάσταση της UI σε κάποια χρονική στιγμή, μέσω ObservableList και Property.
3. **service**: Υπεύθυνες για το business logic της εφαρμογής. Λαμβάνει τα δεδομένα από το repository και τα επιστρέφει στο viewmodel
4. **repository**: Το μοναδικό πακέτο που έχει πρόσβαση στην βάση. Καλεί τις stored procedures που έχουμε δημιουργήσει, με αποτέλεσμα να μην χειριζόμαστε καθόλου SQL μέσα στον κώδικα java
5. **db**: Περιέχει μόνο την DBConnection, δηλαδή διαχειρίζεται την JDBC σύνδεση
6. **model**: 15 java classes οι οποίες λειτουργούν απλά ως DTOs και οι οποίες περιέχουν τα αποτελέσματα των stored procedures

2.3 ΔΙΑΓΡΑΜΜΑ(ΤΑ) ΚΛΑΣΕΩΝ ΚΕΝΤΡΙΚΗΣ ΕΦΑΡΜΟΓΗΣ

Παρακάτω παρουσιάζονται τα διαγράμματα κλάσεων για κάθε πακέτο της εφαρμογής.

1. Controller package:



2. Repository package:

 AuthorRepository

```
   searchAuthors(String, int, int) List<AuthorResult>  
   getAuthorYearStats(int, int, int) List<AuthorYearStat>  
   getAuthorPublications(int, int, int) List<AuthorPublication>  
   getAuthorStats(int) AuthorStats
```

 ChartRepository

```
   getPublisherStats() List<PublisherStat>  
   getCategoryYearStats(String, String) List<CategoryYearStat>  
   getJournalScatter(String) List<JournalScatter>
```

 VenueRepository

```
   getVenueStats(int, String, int, int) VenueStats  
   searchVenues(String, String, int, int) List<VenueResult>  
   getVenueYearDetail(int, String, int, int) List<VenueYearDetail>  
   getVenuePublications(int, String, int, int) List<VenuePublication>  
   getVenueYearStats(int, String, int, int) List<VenueYearStat>
```

 YearRepository

```
   getYearProfile(int) YearProfile  
   getYearPublications(int, String, String, String) List<YearPublication>  
   getPublicationsPerYear(int, int) List<YearStat>
```

3. Service package:

YearService		
 repository		YearRepository
 getYearPublications(int, String, String, String)		List<YearPublication>
 getPublicationsPerYear(int, int)		List<YearStat>
 getYearProfile(int)		YearProfile

ChartService		
 repository		ChartRepository
 getPublisherStats()		List<PublisherStat>
 getCategoryYearStats(String, String)		List<CategoryYearStat>
 getJournalScatter(String)		List<JournalScatter>

VenueService		
 repository		VenueRepository
 getVenueYearStats(int, String, int, int)		List<VenueYearStat>
 getVenueStats(int, String, int, int)		VenueStats
 getVenuePublications(int, String, int, int)		List<VenuePublication>
 getVenueYearDetail(int, String, int, int)		List<VenueYearDetail>
 searchVenues(String, String, int, int)		List<VenueResult>

AuthorService		
 repository		AuthorRepository
 getAuthorStats(int)		AuthorStats
 searchAuthors(String, int, int)		List<AuthorResult>
 getAuthorYearStats(int, int, int)		List<AuthorYearStat>
 getAuthorPublications(int, int, int)		List<AuthorPublication>

4. ViewModel package:

YearViewModel

- toYear *int*
- results *ObservableList<YearStat>*
- publications *ObservableList<YearPublication>*
- loading *BooleanProperty*
- fromYear *int*
- service *YearService*
- yearProfile *ObjectProperty<YearProfile>*
- setFromYear(int) *void*
- setToYear(int) *void*
- getResults() *ObservableList<YearStat>*
- getPublications() *ObservableList<YearPublication>*
- loadYearDetail(int, String, String, String) *void*
- yearProfileProperty() *ObjectProperty<YearProfile>*
- load() *void*
- loadingProperty() *BooleanProperty*

ChartsViewModel

- publisherStats *ObservableList<PublisherStat>*
- categoryStats *ObservableList<CategoryYearStat>*
- scatterData *ObservableList<JournalScatter>*
- loading *BooleanProperty*
- service *ChartService*
- loadScatterData(String) *void*
- getPublisherStats() *ObservableList<PublisherStat>*
- loadingProperty() *BooleanProperty*
- loadPublisherStats() *void*
- getCategoryStats() *ObservableList<CategoryYearStat>*
- getScatterData() *ObservableList<JournalScatter>*
- loadCategoryStats(String, String) *void*

AuthorViewModel

- authorStats *ObjectProperty<AuthorStats>*
- publications *ObservableList<AuthorPublication>*
- searchName *StringProperty*
- toYear *int*
- loading *BooleanProperty*
- fromYear *int*
- service *AuthorService*
- results *ObservableList<AuthorResult>*
- yearStats *ObservableList<AuthorYearStat>*
- search() *void*
- loadingProperty() *BooleanProperty*
- loadProfile(int) *void*
- getResults() *ObservableList<AuthorResult>*
- setToYear(int) *void*
- getPublications() *ObservableList<AuthorPublication>*
- authorStatsProperty() *ObjectProperty<AuthorStats>*
- getYearStats() *ObservableList<AuthorYearStat>*
- setFromYear(int) *void*
- searchNameProperty() *StringProperty*

VenueViewModel

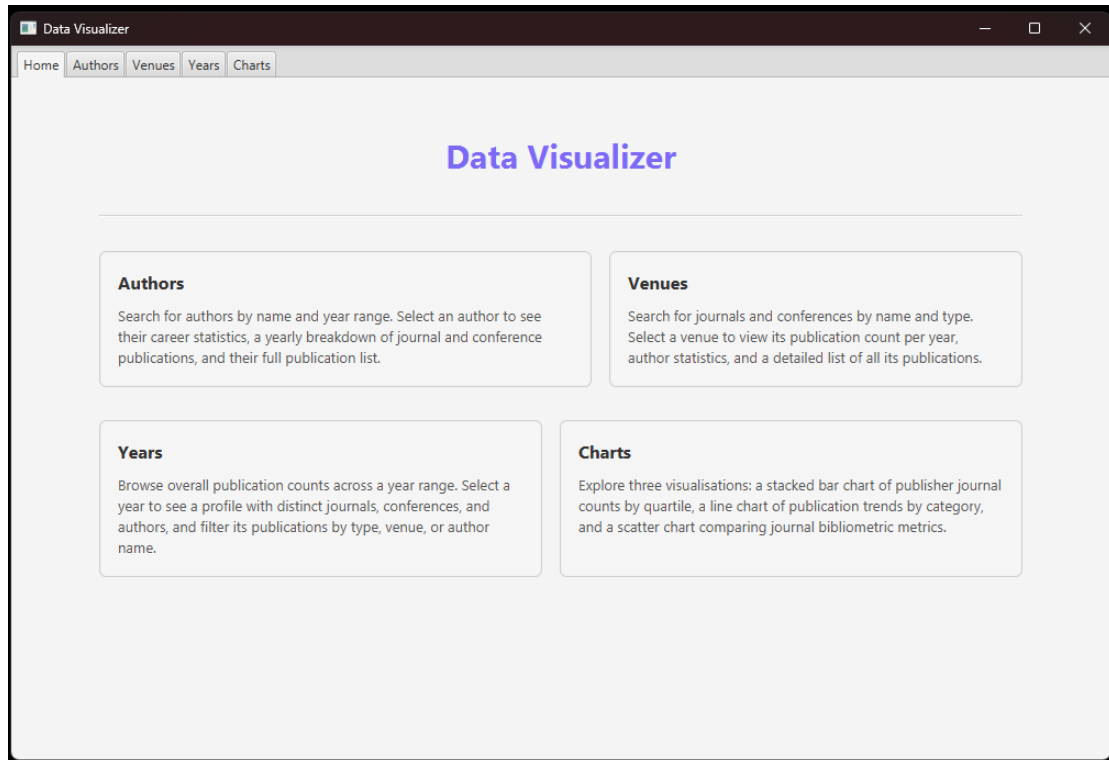
- type *String*
- toYear *int*
- publications *ObservableList<VenuePublication>*
- venueStats *ObjectProperty<VenueStats>*
- fromYear *int*
- loading *BooleanProperty*
- results *ObservableList<VenueResult>*
- searchName *StringProperty*
- service *VenueService*
- yearDetails *ObservableList<VenueYearDetail>*
- loadingProperty() *BooleanProperty*
- getPublications() *ObservableList<VenuePublication>*
- venueStatsProperty() *ObjectProperty<VenueStats>*
- setToYear(int) *void*
- getYearDetails() *ObservableList<VenueYearDetail>*
- setFromYear(int) *void*
- getResults() *ObservableList<VenueResult>*
- searchNameProperty() *StringProperty*
- search() *void*
- loadProfile(int, String) *void*
- setType(String) *void*

5. Model package:



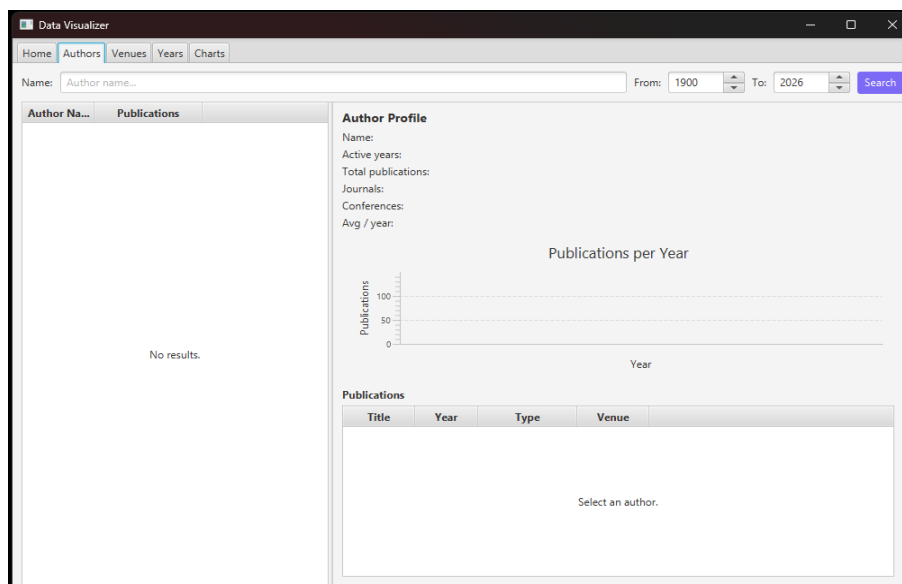
ΔΟΜΗ ΕΦΑΡΜΟΓΗΣ, ΥΠΟΔΕΙΓΜΑΤΑ ΕΡΩΤΗΣΕΩΝ ΚΑΙ ΑΠΑΝΤΗΣΕΩΝ

Αφού τρέξουμε την εφαρμογή, θα εμφανιστεί το Home Page το οποίο έχει την μορφή:

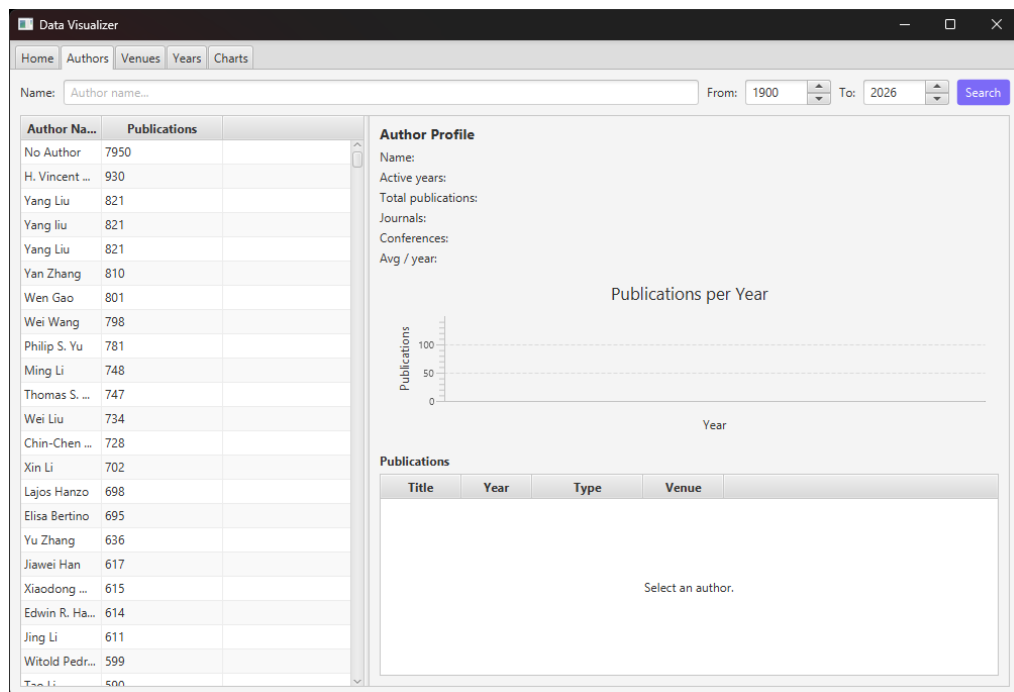


Βλέπουμε ότι πέρα από το home page, έχουμε και 4 άλλα tabs τα οποία θα εμφανίσουν πληροφορίες σχετικά με τους authors, venues, years και κάποια επιλεγμένα διαγράμματα.

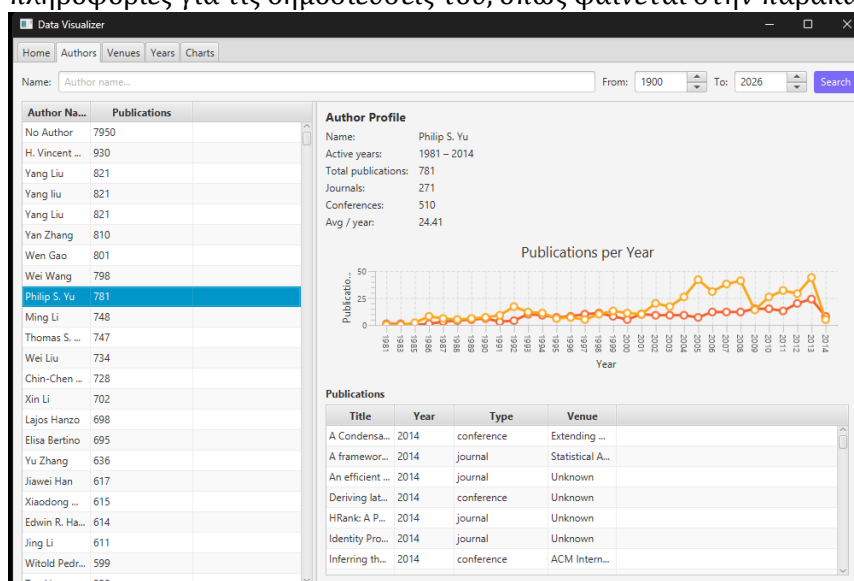
Αν επιλέξουμε το Authors tab θα δούμε αυτό:



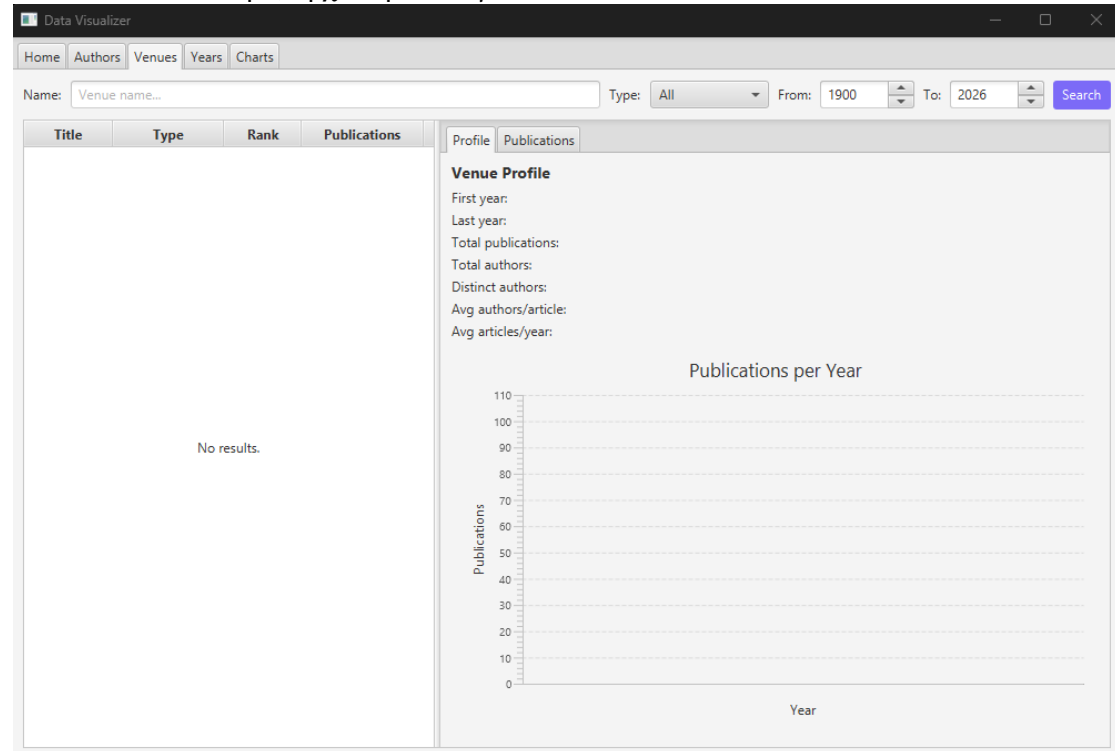
Παρατηρούμε ότι δίνεται η δυνατότητα στον χρήστη να αναζητήσει συγγραφείς με το όνομα τους, μέσα σε ένα όριο ετών στα οποία έχουν δημοσιεύσει. Αν δεν επιλεγεί κάποιο όνομα, τότε θα εμφανιστούν όλοι οι συγγραφείς που έχουμε στην βάση μας. Πατώντας το Search και αφού περιμένουμε για λίγο θα εμφανιστεί αυτό:



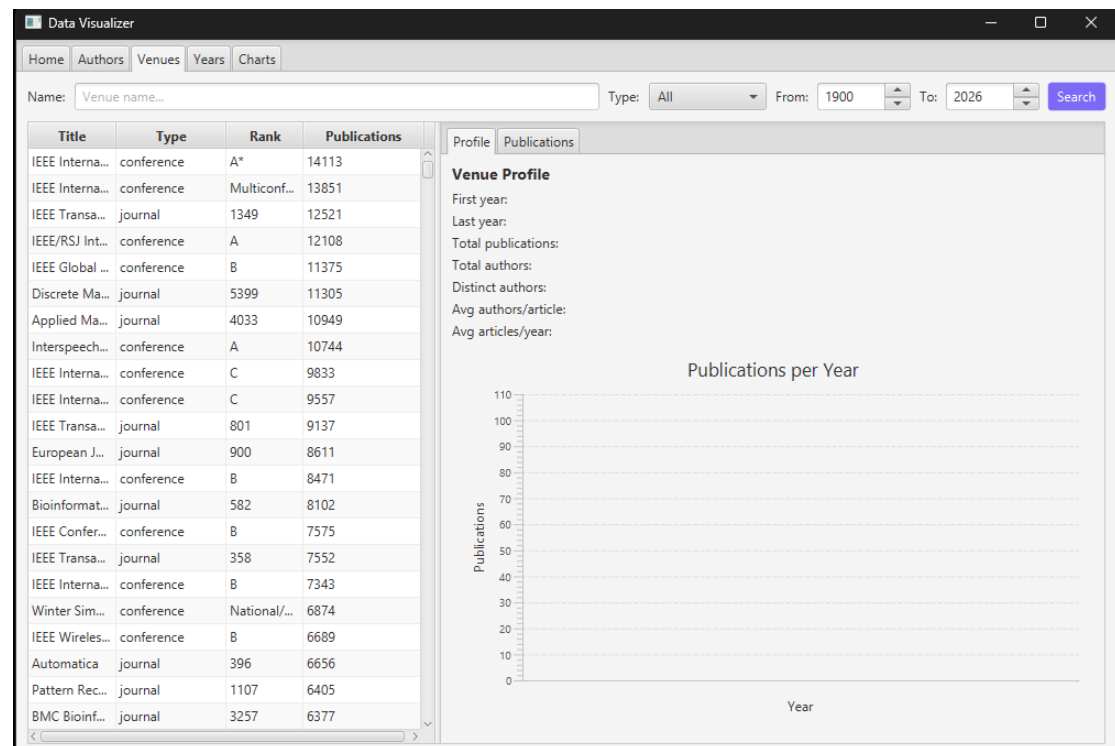
Στην οθόνη είναι όλοι οι συγγραφείς που έχουμε στην βάση και ο αριθμός των δημοσιεύσεων κάθε ένα από αυτούς. Με No Author έχουμε αποθηκεύσει όλες εκείνες τις publications που δεν είχαν matching author. Αν κάνουμε κλικ πάνω σε έναν συγγραφέα, μπορούμε να δούμε διάφορες πληροφορίες για το προφίλ του, όπως την 1^η και την τελευταία χρονιά που δημοσιεύσε, τον αριθμό των διακριτών περιοδικών/συνεδρίων, αλλά και τον μέσο αριθμό άρθρων ανά χρονιά. Επίσης μας παρουσιάζονται και πληροφορίες για τις δημοσιεύσεις του, όπως φαίνεται στην παρακάτω εικόνα:



Στο Venues tab τώρα, αρχικά βλέπουμε αυτό:

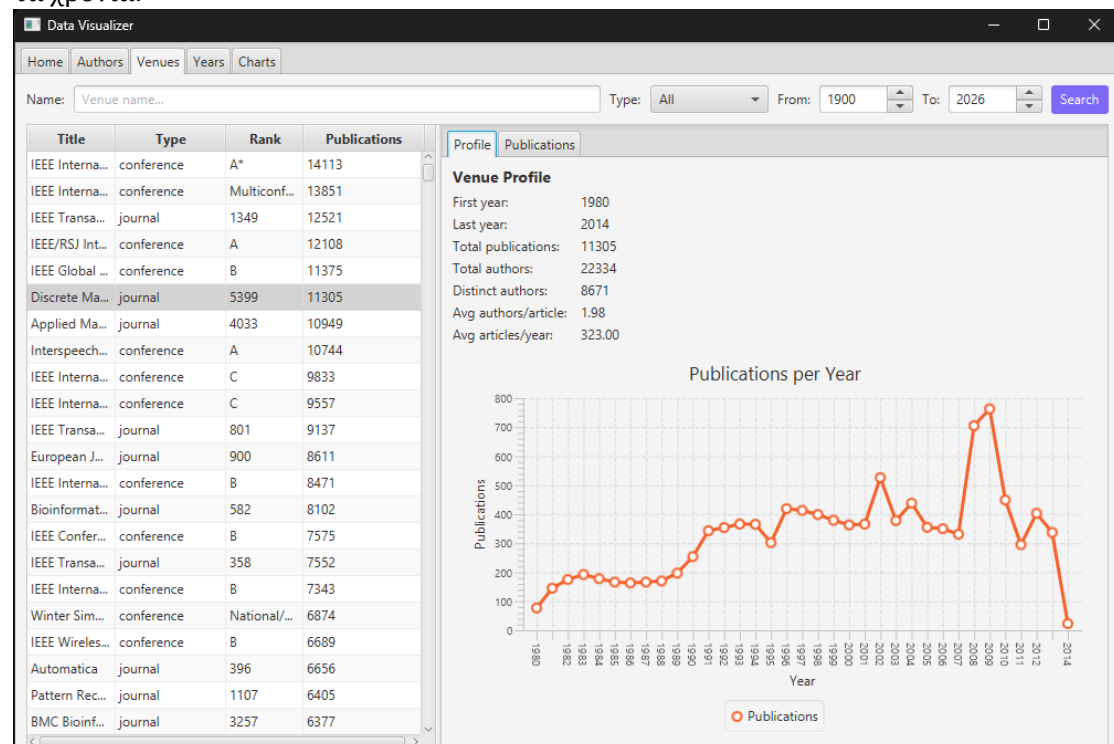


Όπως στους συγγραφείς, έτσι και εδώ μπορούμε να αναζητήσουμε κάποιο συνέδριο/περιοδικό με το όνομα του, τον τύπο του και την χρονία που εμφανίστηκε. Έστω ότι πατάμε το Search, αφού περιμένουμε λίγο θα δούμε αυτό:



Εδώ εμφανίζονται όλα τα περιοδικά/συνέδρια που έχουμε στην βάση μας με πληροφορίες όπως τίτλος, τύπος, κατάταξη και αριθμό δημοσιεύσεων. Αν κάνουμε κλικ σε ένα τυχαίο απο αυτά, θα δούμε διάφορες πληροφορίες για αυτό το

συνέδριο/περιοδικό όπως πρώτη και τελευταία χρονιά που εμφανίστηκε, συνολικό αριθμό δημοσιεύσεων και συγγραφέων, μέσους όρους συγγραφέων και άρθρων καθώς και ένα διάγραμμα που δείχνει την μεταβολή των δημοσιεύσεων σε αυτό το άρθρο ανά τα χρόνια:



Αν κάνουμε κλικ στο πεδίο Publications θα δούμε πληροφορίες για τις δημοσιεύσεις που έγιναν σε αυτό το περιοδικό:

The screenshot shows the 'Data Visualizer' application with the 'Publications' tab selected. The table displays a list of publications for the selected venue, including the title, year, and number of authors.

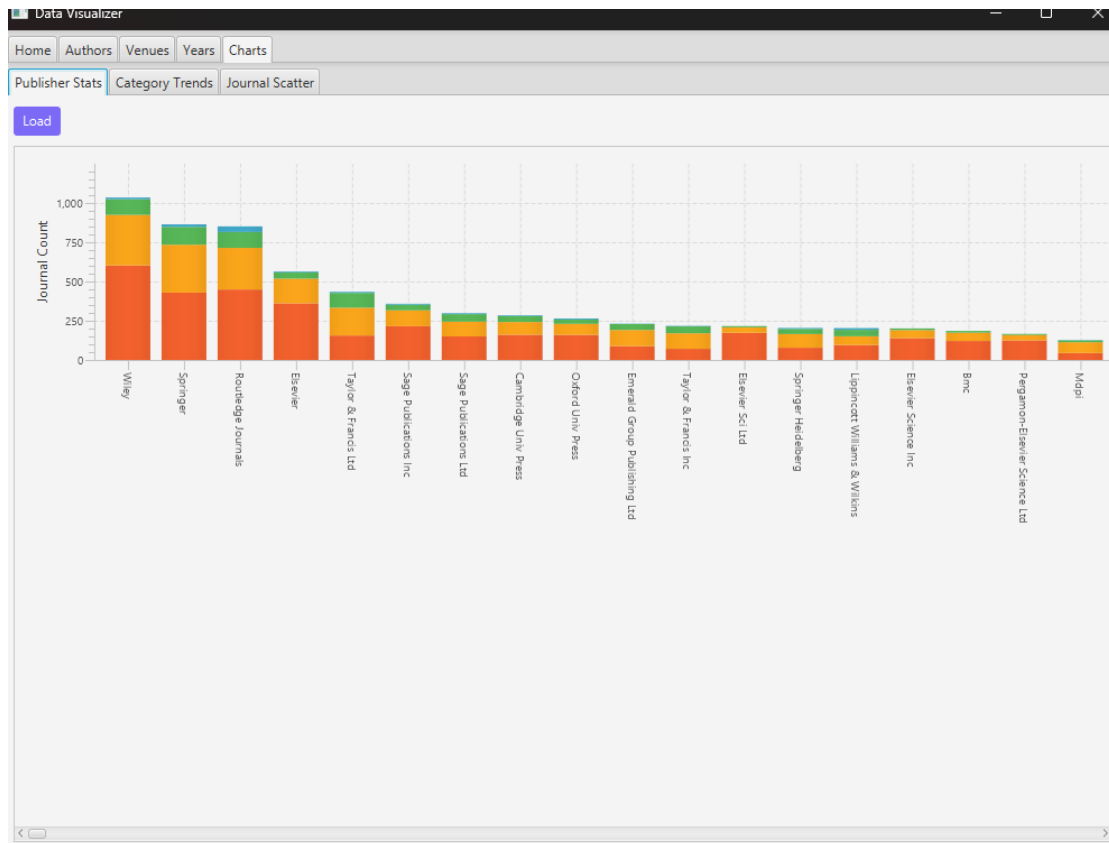
Title	Year	Authors
A new char...	2014	1
A note on "...	2014	1
A short not...	2014	1
A weight st...	2014	2
An evasion ...	2014	1
Chebyshev ...	2014	1
Codes asso...	2014	3
Critical sets ...	2014	2
Every 3-pol...	2014	3
Extremal m...	2014	3
Hamiltonia...	2014	2
Improved b...	2014	2
Incidence m...	2014	3
Induced ma...	2014	2
Injective col...	2014	2
New transfe...	2014	3
On Hamilto...	2014	1
On r-hued c...	2014	5
Outerplanar...	2014	3
Permutatio...	2014	15
Proper cate...	2014	2

Στο Years tab τώρα, μπορούμε να αναζητήσουμε πληροφορίες για μια συγκεκριμένη χρονιά (ή όλες αν δεν βάλουμε κάποιο φίλτρο):

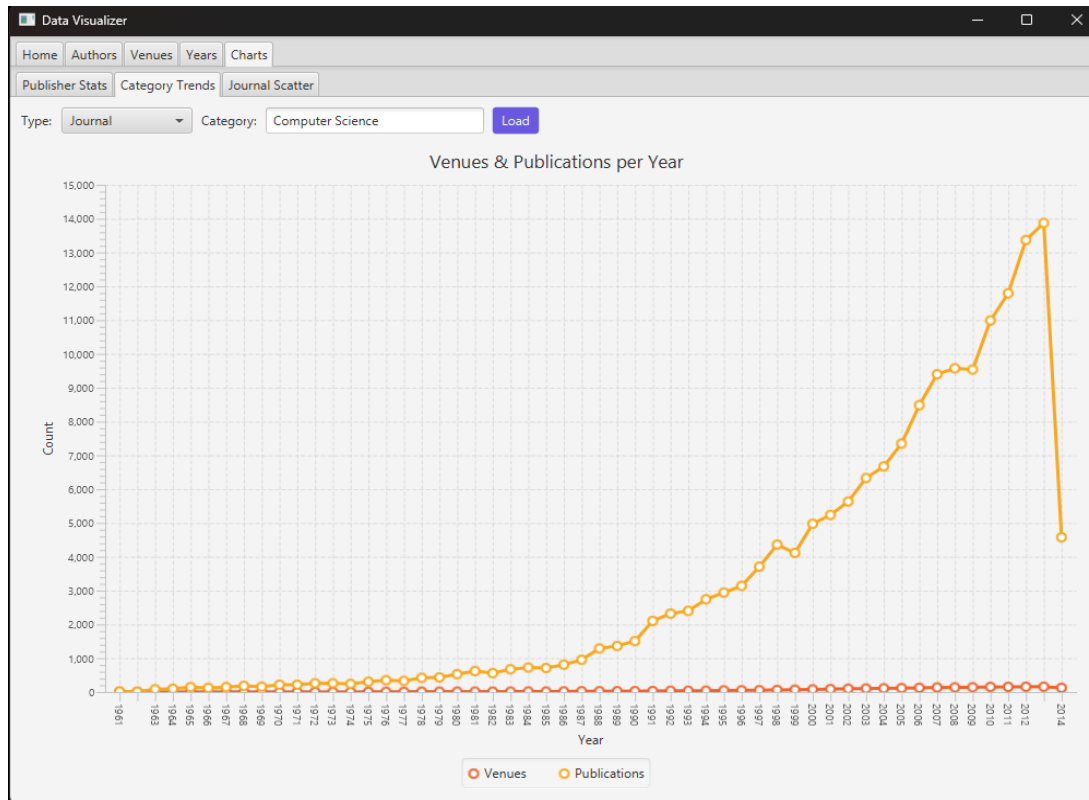
Πατάμε το show και κάνουμε κλικ σε μια τυχαία χρονιά και βλέπουμε αυτό:

Ουσιαστικά μπορούμε να δούμε το προφίλ μιας χρονιάς, τις συνολικές δημοσιεύσεις, τον αριθμό των περιοδικών/συνεδριών που έλαβαν μέρος αυτή την χρονιά, τους διαφορετικούς συγγραφείς, αλλά και πληροφορίες για τις αντίστοιχες δημοσιεύσεις (τις οποίες μπορούμε και να φιλτράρουμε).

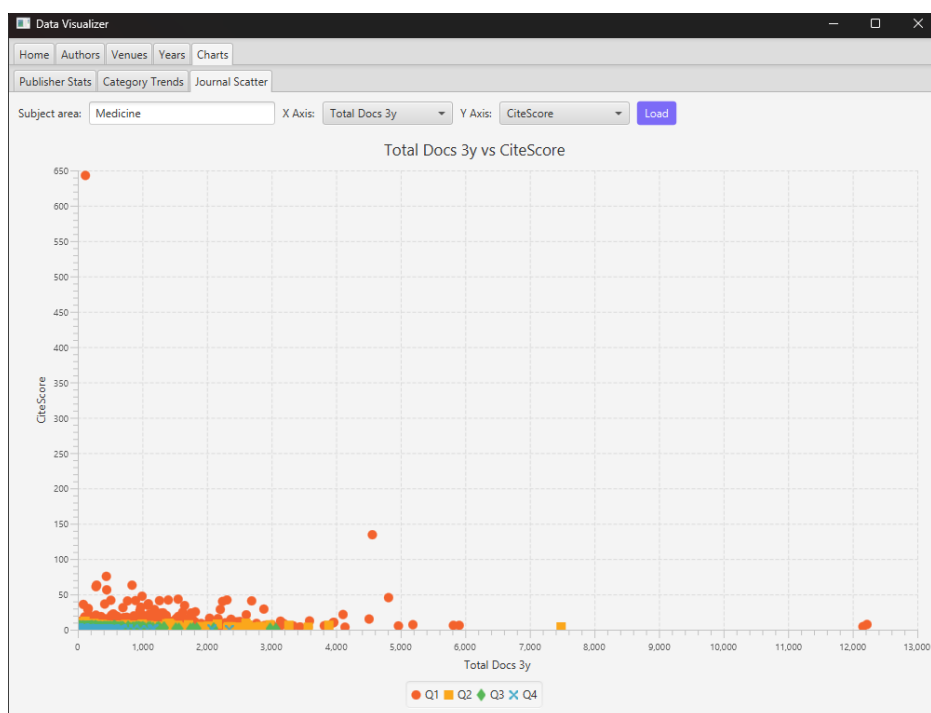
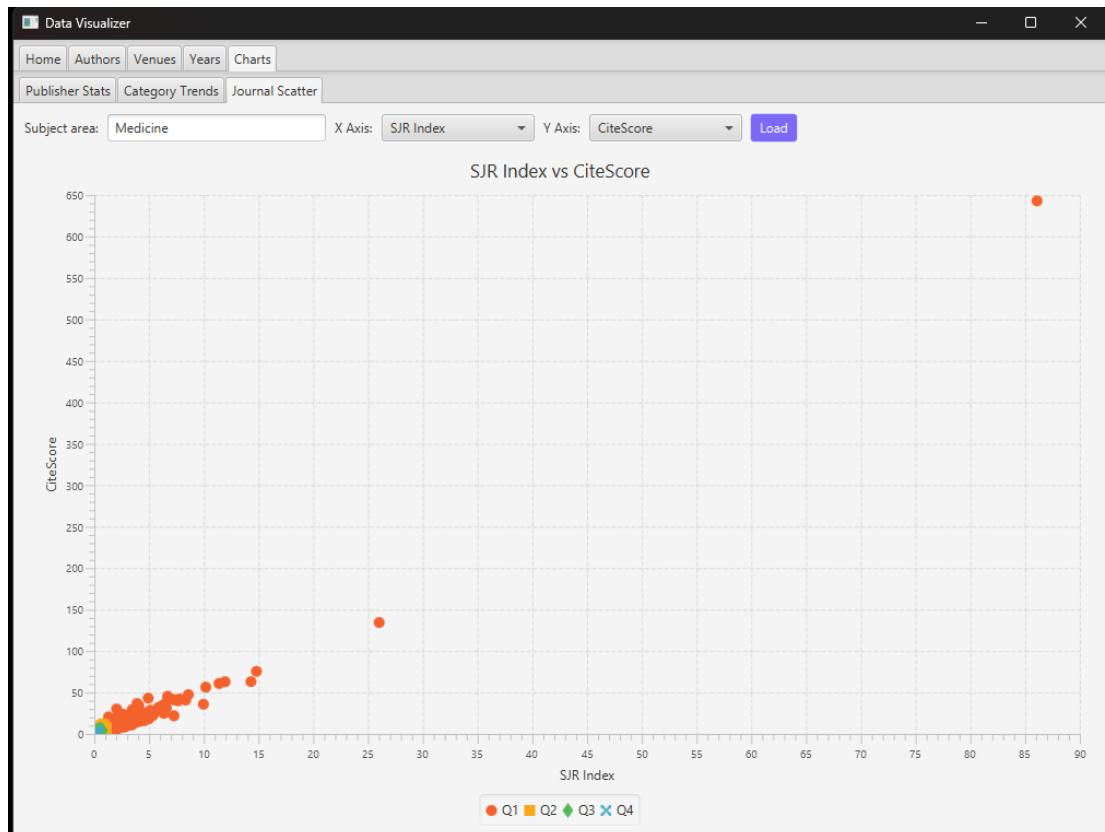
Τέλος στο Charts tab έχουμε 3 υπό-tabs. Το 1ο είναι το Publisher Stats όπου πατάμε Load και εμφανίζεται ένα Stacked Bar Chart με τους publishers στον οριζόντιο άξονα και τον αριθμό των journals στον κατακόρυφο. Κάθε μπάρα είναι χωρισμένη σε 4 χρώματα που αντιστοιχούν στα quartiles (Q1, Q2, Q3, Q4), οπότε βλέπουμε άμεσα πόσα journals έχει κάθε publisher ανά quartile.



Στο 2ο υπό-tab, Category Trends, επιλέγουμε αν θέλουμε Journals ή Conferences από το dropdown, γράφουμε μια κατηγορία (π.χ. Computer Science) και πατάμε Load. Εμφανίζεται ένα Line Chart με δύο γραμμές, μια για τον αριθμό των venues και μια για τον αριθμό των δημοσιεύσεων ανά έτος, ώστε να δούμε την εξέλιξη της κατηγορίας στον χρόνο.

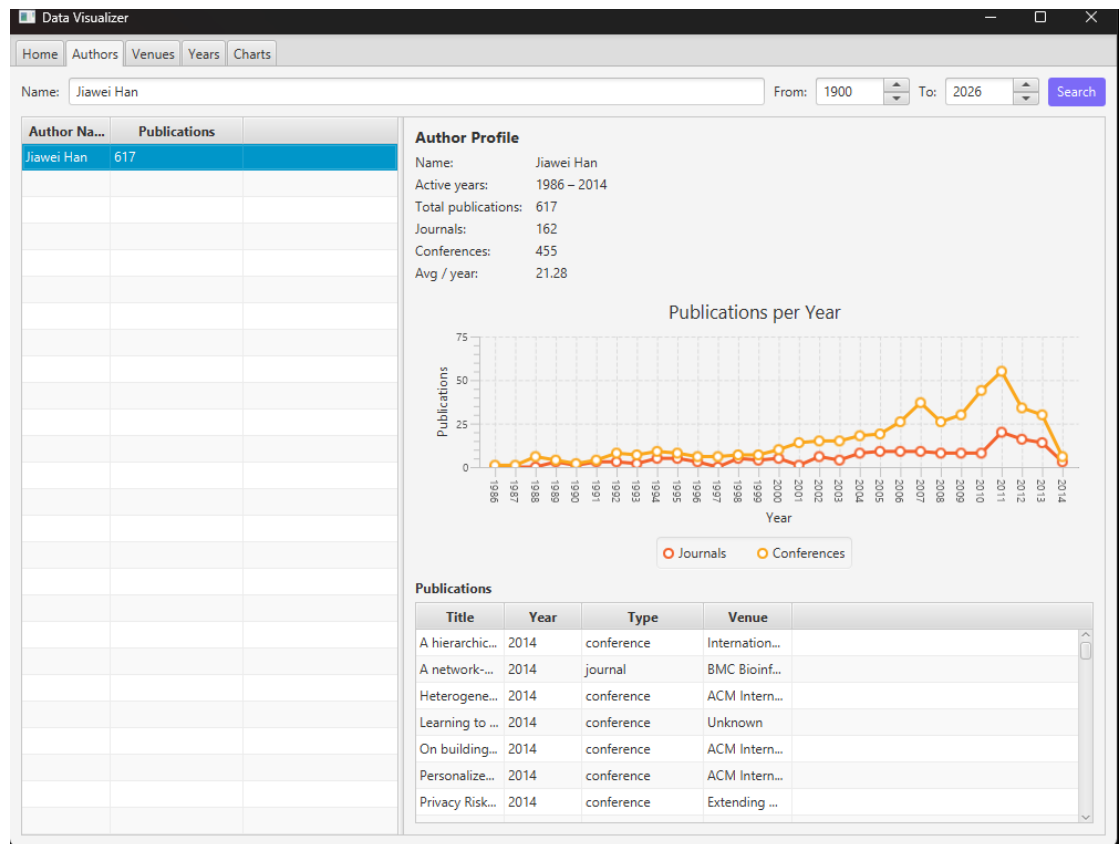


Στο 3ο υπό-tab, Journal Scatter, γράφουμε κάποιο subject area (π.χ. Medicine), επιλέγουμε τι θέλουμε στον Χ και Υ άξονα από τα διαθέσιμα metrics (SJR Index, CiteScore, H-Index κ.α.) και πατάμε Load. Κάθε σημείο του Scatter Chart αντιστοιχεί σε ένα journal, χρωματισμένο ανάλογα με το quartile του (Q1/Q2/Q3/Q4). Έτσι μπορούμε να συγκρίνουμε οποιονδήποτε συνδυασμό μετρικών για τα journals ενός συγκεκριμένου πεδίου.



Κάποια παραδείγματα ερωτήσεων και ο τρόπος που μπορούν να απαντηθούν:

1. Ποια δημοσίευση έκανε ο συγγραφέας Jiawei Han και πόσες συνολικά έχει ?



2. Πόσες δημοσιεύσεις έχει το περιοδικό Bioinformatics ανά έτος ?

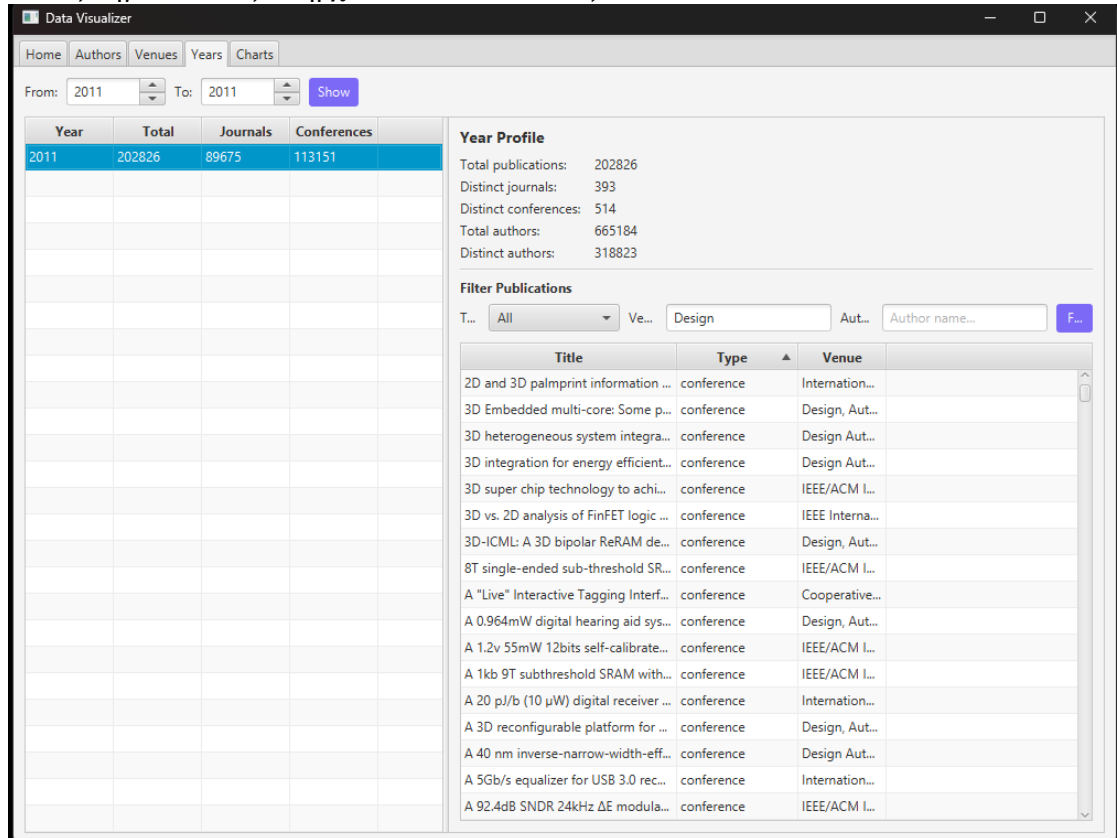
Publications

Title	Type	Rank
Bioinformatics	journal	582
BMC Bioinformatics	journal	3257
IEEE Bioinformatics and Bioengi...	conference	C
Briefings in Bioinformatics	journal	1025
Journal of Bioinformatics and Co...	journal	12941
IEEE Symposium on Computatio...	conference	C
Workshop on Algorithms in Bioi...	conference	C
Asia-Pacific Bioinformatics Conf...	conference	B
Journal of integrative bioinform...	journal	6346
International Conference on Pra...	conference	National...

Publications

Title	Year	Authors
A novel sig...	2014	7
A novel spli...	2014	4
A pathway...	2014	4
A penalized...	2014	2
A phylogen...	2014	1
A poor man...	2014	2
A probabilis...	2014	4
A statistical ...	2014	3
A user-orie...	2014	3
A wavelet-b...	2014	3
ABACUS: an...	2014	4
Accessibility...	2014	4
Accurate id...	2014	3
Adaptive re...	2014	3
AISALC: a so...	2014	11
AllergenFP: ...	2014	4
Anatomical ...	2014	2
AnchorMS: ...	2014	2
Andreas Dr...	2014	1
Assimilating...	2014	2
ASSIST: a fa...	2014	5
ATHENA: th...	2014	5
Automated ...	2014	8
Automated ...	2014	8
AVIA: an int...	2014	3

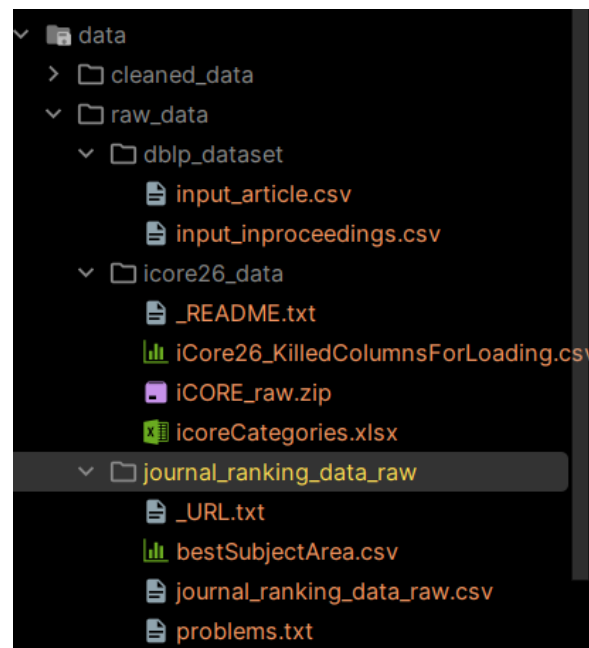
3. Πόσες δημοσιεύσεις υπήρχαν συνολικά το έτος 2011 ?



3 ΛΟΙΠΑ ΣΧΟΛΙΑ

Επειδή στο github, κάποιος θα ήθελε μόνο τα cleaned data, ώστε να τα βάλει στο φάκελο uploads της MySQL και να είναι έτοιμος να τρέξει την εφαρμογή, σας παραθέτω εδώ την δομή που πρέπει να έχει το project ώστε να τρέξετε αν θέλετε την όλη διαδικασία ETL.

Μέσα στο app/data/ θα πρέπει η δομή να είναι ως εξής:



Ανοίγουμε το αρχείο transformations/prepare_and_clean_data.kjb στο Pentaho Spoon και το τρέχουμε. Τα transformations χρησιμοποιούν τις παραμέτρους RAW_DATA_PATH και CLEANED_DATA_PATH που δείχνουν αυτόματα στους σωστούς φακέλους σχετικά με τη θέση του αρχείου, οπότε δεν χρειάζεται καμία αλλαγή. Μετά την επιτυχή εκτέλεση, τα 5 cleaned CSV αρχεία θα εμφανιστούν αυτόματα στον φάκελο app/data/cleaned_data/



και είναι έτοιμα για φόρτωση στη MySQL σύμφωνα με τα βήματα που περιγράφονται στο README.